

DSU CAST AUTONOMOUS RESEARCH PROGRAM

Week 1 Capstone — Spot Patrol Data Analyzer

Assigned: Monday June 15 | Demo Day: Monday June 22 | Individual project

What You Are Building

Spot just completed three patrol laps around the CAST building. Its onboard sensors recorded 792 readings across 11 waypoints — temperature, battery level, motor heat, proximity, and movement speed.

That data is sitting in a CSV file. Nobody has looked at it yet. Your job is to write a Python script that reads the file, analyzes it, finds what went wrong during the mission, and writes a summary report.

THE POINT

When you work with a real Spot robot, this is the exact workflow: the mission runs, it produces a log, you analyze the log to understand what happened. You are building the analysis tool.

Your Data — spot_patrol_data.csv

You have been given a file called `spot_patrol_data.csv`. Drop it into your `/workspace/` directory inside the Docker container. It has 792 rows and 8 columns.

Column Reference

Column	Example	What It Means
<code>timestamp</code>	2026-06-29 09:00:04	When this reading was recorded
<code>lap</code>	1	Which patrol lap (1, 2, or 3)
<code>waypoint</code>	lab_101	Where Spot was when the reading was taken
<code>reading_num</code>	2	Which reading at this waypoint (1, 2, or 3)
<code>sensor_id</code>	temp_motor_FL	Which sensor produced the reading
<code>value</code>	47.83	The sensor reading value
<code>unit</code>	C	Unit of measurement (C, %, cm, or m/s)
<code>status</code>	OK	OK, FAULT_HIGH, or FAULT_LOW

The Sensors

- **temp_body**: Spot's body temperature in Celsius. Normal range: 28–38 C
- **temp_motor_FL / FR / RL / RR**: Front-left, front-right, rear-left, rear-right leg motor temperatures. Normal range: 30–55 C

- **battery_pct:** Battery charge percentage. Goes down over the mission. Below 15% is a fault.
- **proximity_cm:** Distance to the nearest object in centimeters. Below 10 cm is a fault.
- **velocity_ms:** Spot's movement speed in meters per second. Normal range: 0–1.6 m/s

HEADS UP

Something went wrong during this mission. The data contains real faults. Part of your job is to find them — don't assume all 792 rows say OK.

Your Tasks

Build one Python script called `analyzer.py`. It should complete all five tasks below, in order, when run. Each task builds on the previous one.

REQUIREMENTS

Your script must run inside the Docker container and be triggerable over SSH with a single command.

Every file operation must use `try/except`.

All analysis logic must live inside functions — nothing floating at the top level.

All output goes to a report file AND prints to the terminal.

1

Load the Data

Read `spot_patrol_data.csv` using Python's built-in `csv` module. Store each row as a dictionary in a list. Print how many rows were loaded.

Hint: Use `csv.DictReader` — it gives you each row as a dictionary automatically, which is exactly the data structure you need.

2

Mission Summary

Print a summary of the mission: how many laps, how many unique waypoints visited, how many total sensor readings, and how many unique sensor types. These numbers should come from the data — don't hardcode them.

Hint: Use a Python `set()` to find unique values. `len(set())` gives you the count of unique items in a list.

3

Find All Faults

Find every row where status is not OK. For each fault, print the timestamp, waypoint, sensor_id, value, unit, and status. Then print the total fault count at the end.

Hint: Loop through your list of row dicts. Check if `row['status'] != 'OK'`. You should find 13 faults — if you find a different number, check your loading logic.

4

Per-Sensor Averages

For each unique sensor_id, calculate the average reading value across all laps and all waypoints. Print the results sorted from highest average to lowest. Round all averages to 2 decimal places.

Hint: Use a dictionary where the key is `sensor_id` and the value is a list of readings. Then calculate the average of each list. To sort a dictionary by value, look up `sorted()` with a `key=` argument.

5

Write a Report File

Write all of the above output into a file called `mission_report.txt` in your `/workspace/` directory. The report should have clear section headers and be readable by someone who wasn't in the room. Include a timestamp at the top showing when the report was generated.

Hint: Open the file with `open('mission_report.txt', 'w')` inside a `with` block. Write each section the same way you printed it — just replace `print()` with `file.write()`. Don't forget newlines (`\n`).

How to Run and Submit

Step 1 — Drop the data file in your workspace

```
cp /path/to/spot_patrol_data.csv /workspace/spot_patrol_data.csv
```

Step 2 — Run your script inside the container

```
python3 /workspace/analyzer.py
```

Step 3 — Trigger it over SSH from your host machine

This must work from outside the container. If it doesn't, your submission is incomplete.

```
ssh root@localhost -p 2222 'python3 /workspace/analyzer.py'
```

Step 4 — Push to GitHub

- Create a repo named: `cast-capstone-[yourname]`
- Push `analyzer.py` and `mission_report.txt`
- Your README must include: what the script does, how to run it, and what faults it found
- Post the repo link in Teams by 5 PM Friday June 27

Demo Day — Monday June 22

Demo Day is 5 minutes per intern. No slides. No screenshots. Just the terminal.

WHAT YOU WILL SHOW

1. Trigger the script over SSH from your host machine — one command, live.
2. Show the output printing to the terminal as it runs.
3. Open `mission_report.txt` and show the contents.

4. Point to the faults section and explain what went wrong on the mission.
5. Show your GitHub repo and commit history.

Tariq will ask you one question during your demo. It will be some version of this:

THE QUESTION

"If this was a real Spot robot instead of a CSV file, what would you change in your code to make this work?"

You do not need to know the Spot SDK to answer this. You just need to understand what your own code is doing and where the data comes from. Think about it before Demo Day.

How You Will Be Evaluated

What we look at	Not there yet	Ready for the robot
All 5 tasks complete	One or more tasks missing or producing wrong output	All 5 tasks run and produce correct output — the 13 faults are found, averages are accurate, report is written
Code structure	Logic floating at the top level, no functions, hard to follow	All logic in named functions, main execution block at the bottom, easy to read
Error handling	No try/except — script crashes if the file is missing or malformed	File open and read wrapped in try/except with a clear error message
SSH trigger	Script requires manual steps inside Docker to run	Single SSH command from the host machine runs the full script cleanly
Report file	No report file, or report is just a raw data dump with no structure	Report is clearly formatted with section headers, readable by someone new to the data
Demo + explanation	Cannot explain what the script is doing or what the faults mean	Can walk through the output live and explain what went wrong during the mission

If You Get Stuck

- Re-read the task description carefully. Most confusion comes from not reading closely enough.
- Print intermediate values. If you don't know what's in your data structure, print it.
- Post the exact error message in Teams — not a description of the error, the actual text.
- The csv module documentation at docs.python.org is the best reference for Task 1.
- You built a sensor data pipeline in Day 4 of Week 1. That code is your starting point for this.

REMINDER

You already did the hard part of this in Week 1. You read files, stored data in dicts, looped through lists, and wrote output files. This capstone is those same skills applied to real robot data.